

Software Engineering

Declarative prototyping - Exercises -

Exercise 1

- Quicksort with difference lists – declarative prototype:

```
qsd :: [Int] -> [Int] -> [Int]
qsd []      ys = ys
qsd (x : xs) ys =
    let (ls,gs) = partit x xs
    in qsd ls (x : qsd gs ys)
```

```
partit :: Int -> [Int] -> ([Int],[Int])
partit p []      = ([],[ ])
partit p (x:xs) =
    let (ls,gs) = partit p xs
    in  if (x < p) then (x : ls,gs) else (ls,x:gs)
```

```
Main> qsd [1,7,3,12,4,2,5] [0,0,0]
[1,2,3,4,5,7,12,0,0,0]
```

Exercise 2

- Binary search tree merging using difference lists:

```
data Tree = NIL | T Tree Int Tree
flatt :: Tree -> [Int] -> [Int]
flatt NIL      ys = ys
flatt (T l n r) ys = flatt l (n : flatt r ys)

merge :: Tree -> Tree -> [Int] -> [Int]
merge NIL t ys = flatt t ys
merge t NIL ys = flatt t ys
merge (T NIL n1 r1) (T NIL n2 r2) ys =
    if (n1 < n2) then n1:merge r1 (T NIL n2 r2) ys
                  else n2:merge (T NIL n1 r1) r2 ys
merge (T (T l11 ln1 lr1) n1 r1) t2 ys =
    merge (T l11 ln1 (T lr1 n1 r1)) t2 ys
merge t1 (T (T l12 ln2 lr2) n2 r2) ys =
    merge t1 (T l12 ln2 (T lr2 n2 r2)) ys

t1 = T (T NIL 1 (T NIL 3 NIL)) 5 (T NIL 7 NIL)
t2 = T (T NIL 2 NIL) 4 (T (T NIL 6 NIL) 8 NIL)
Main> merge t1 t2 [0,0]
[1,2,3,4,5,6,7,8,0,0,0]
```

Exercise 3

- Lisp-like structure flattening – declarative prototype:

```
data Lisp = Nil | Atom Int | Cons Lisp Lisp
```

```
flat :: Lisp -> [Int] -> [Int]
```

```
flat Nil ys = ys
```

```
flat (Atom n) ys = n:ys
```

```
flat (Cons car cdr) ys = flat car (flat cdr ys)
```

```
lisp :: Lisp
```

```
lisp = Cons (Cons (Cons (Atom 1) Nil)  
              (Cons Nil (Atom 2)))  
          (Cons (Atom 3) (Atom 4))
```

```
Main> flat lisp [0,0,0]
```

```
[1,2,3,4,0,0,0]
```